Exercise sheet 1[1]

Coding exercises

Q1.1 Compile and run the following C++ program, which displays the text `"Hello, world !"`.

```
#include <iostream>
int main()
{
 std::cout << "Hello, world!" << std::endl;
 return 0;
}
```

Q1.2 In the following code snippet, double-precision variables `a` and `b` are declared, and initialised with the values 5 and 3 respectively.

```
int main()
{
 double a = 5.0;
 double b = 3.0;
 return 0;
}
```

Complete the program so that it displays the values of `a`, `b` and their sum. For example, the output of your program could be:

```
The value of 5 + 3 is 8.
```

You will want to add at least two (probably three) new statements to the program.

Q1.3 Write a program that calculates and displays the area and circumference of a circle of radius 5.

Note that C++ does not have a definition of π so you will have to define a variable to store $\pi = 3.14159\ldots$ yourself. What type should this variable be, and how many digits of π can it store?

Modify your program so that it displays the area and circumference for circles of radius $0, 1, 2, \ldots, 10$.

Q1.4 Write a program to calculate the nth Fibonacci number. The Fibonacci numbers f_n are defined by the sequence

$$f_0 = 1, \tag{1}$$

$$f_1 = 1, \tag{2}$$

$$f_n = f_{n-1} + f_{n-2} \quad \text{for} \quad n \geq 2. \tag{3}$$

[1] Version 28/09/2026.
Feedback, comments, corrections, applause, etc. to: `M.Heil@maths.manchester.ac.uk`

28/09/2026, 07:59:00

Think carefully about how to perform one iteration of this sequence within a loop. You do *not* need to store all the previous values! What's the minimum number of values you need to keep track of as the iteration proceeds?

Note: We use a slightly extended version of this question as a model for the analysis questions that will feature on each example sheet.

28/09/2026, 07:59:00

`int` and `double` arithmetic

In the lecture videos, we covered the concept of *operators* (such as +,−,∗,/), which can operate on variables. For the mathematical operations performed on `double` variables, the results are usually as we would expect. However, we can also perform these operations on `int` (integer) variables, and in this case, there are some subtleties.

If both the operands of one of the mathematical operators (such as +,−,∗,/) are of the same type (e.g. of type `int`), then the result of evaluating that operator will be of that same type. This means that when the division operator `/` is used on two `int` variables, it produces an **integer** result, even when the numerator is not exactly divisible by the denominator. When two operands of type `int` are divided, the result (if non-integer) will always be rounded towards zero to the next integer (e.g. 3.6 is rounded to 3, −3.6 is rounded to −3). Note that this not necessarily the closest integer.

This behaviour is particularly difficult to spot when the operands of the division operator are literals, i.e. if they are just a number, rather than the name of a variable. Numeric literals are considered to be of type `int` if they do not include a decimal point, so `3`, `-15`, etc. are all of type `int`. To obtain a literal that is of type `double`, use a decimal point, e.g. `3.0`, `-15.0`, `5.65`. For `double` literals that are in fact integers, the zero after the decimal point is not needed, so `3.` or `3.0` can be used to express a `double` variable with the value 3.

If either of the operands of an operator is a `double`, the other operand will be converted (or 'promoted') to the `double` type before the division takes place. That means that if either operand of the division operator `/` is a `double`, the result of the division will be of type `double`, and can be a fractional value.

A second subtle behaviour of `int` variables is in their maximum and minimum values. The largest number that a `int` can store is usually $2^{31} - 1 \approx 2 \times 10^9$. If an `int` variable is incremented by one beyond this, it 'wraps' to the smallest supported number $-(2^{31} - 1)$. This behaviour is usually undesirable, so integers should be kept within their maximum and minimum ranges. The `unsigned` data type is similar to `int` but stores natural numbers (non-negative integers). Although a `unsigned` variable cannot store negative numbers, it can store positive integers up to a larger maximum limit, $2^{32} - 1 \approx 4.2 \times 10^9$.

Q1.5 What values do you expect to be stored in the variables `a` to `k`, and why?

```
// Define some variables
double x = 5.0, y = 4.0;
int m = 5, n = 4;

// Now compute with them
double a = 4/3;
double b = 3/4;
double c = 1.0/2;
double d = 1/2.0;
double e = 1.0/2.0;
int f = 1.0/2.0;
```


28/09/2026, 07:59:00

```
double g = x/y;
double h = x/n;
double i = m/n;
double j = m/4;
double k = m/4.0;
```

Write a program to check whether your expectations were correct.

Q1.6 The following code has several bugs (at least 5)

```
#include <iostream>
#include <cstdlib>

// Program to calculate the sum:
//  1^2 + 2^2 + 3^2 + ... + 2000^2
//
// The result should be 2,668,667,000
int main()
{
 int n=2000;
 int sum=0;

 i = 1;
 while (i<n)
  {
   sum += i*i
   i++;
  }

 std::cout << "The sum of the first " << n <
  " squares is " << sum << std::endl;
 return 0;
}
```

Some of these are highlighted as errors by the compiler, but some may only become evident once the program has been run. Fix the errors.

28/09/2026, 07:59:00

Analysis Exercise

The Analysis Exercises and how they're assessed

The first four exercise sheets contain a short "Analysis Exercise" which will ask you to

1. write C++ code that performs specified tasks,
2. explore the results and, where appropriate, perform some accompanying analysis,
3. summarise the results (and the analysis) concisely in a "powerpoint"-style presentation.

The tasks will be assessed on GRADESCOPE. You will have to upload (i) your C++ source code and (ii) the pdf file of your presentation to the course's GRADESCOPE page, accessible via Canvas. To ensure conciseness, the pdf file will have a page limit. GRADESCOPE will then perform some automated tests on your code, allocate grades and provide diagnostics. Before the deadline you can upload the files as many times as you want, and thus deal with any problems that are reported. The final grade for the exercise will be based on the most recent submission received before the deadline.
The code and the report will be subject to a light-touch review, the spirit being very much like the first-year undergraduate tutorials: we primarily want to see that you've had a go at the question, and will briefly look at the source code and the pdf file. If it contains a sensible attempt you'll get full marks; if things look sloppy you'll get half marks; if there's no file (or it's just some dummy file) you'll get *nul point* as the Germans don't say.
The main aim of our looking at the files is to get a feeling for common errors and to spot room for improvement. We will briefly discuss our own "model" solution and contrast it with what we've seen in the students' submissions in the following computer lab class. It's important to stress that there's no one right way to do things – this applies to writing code and reports! Some of it is a matter of style, but there may be ways to do things objectively better.
While these exercises don't carry many marks (see below), they act as a "warm up" for the two bigger projects (worth 30% and 60% respectively) to be done later. These projects will be assessed in essentially the same way, but with more attention paid to the report (and more specific instructions given). Hence the main reward for engaging with these exercises is that you will have a much better feeling for what is expected. If you want any more detailed comments on your code and report, please ask during the lab classes.
To give you some guidance, the Canvas page provides the model solutions (C++ code, the pdf file for the report, and the latex source code used to generate it) for an extended version of a question from Exercise Sheet 1. Have a look and experiment with uploading the files to GRADESCOPE (though you obviously won't get any marks for this!).

28/09/2026, 07:59:00

Analysis Exercise 0: The Fibonacci sequence

1. Write a C++ code `analysis_exercise0.cpp` that computes the first 16 Fibonacci numbers f_n ($n = 0..15$), defined by the sequence

$$f_0 = 1, \tag{4}$$
$$f_1 = 1, \tag{5}$$
$$f_n = f_{n-1} + f_{n-2} \quad \text{for} \quad n \geq 2. \tag{6}$$

   Think carefully about how to perform one iteration of this sequence within a loop. You do *not* need to store all the previous values! What's the minimum number of values you need to keep track of as the iteration proceeds?

2. Plot f_n as a function of n.

3. Validate your code by checking that the computed numbers actually satisfy the specified relation, and by comparing them against the explicit expression for the Fibonacci numbers, given, e.g., on Wikipedia at

   `https://en.wikipedia.org/wiki/Fibonacci_sequence`

The final version of your code should output (on the screen, using `std::cout`) 16 lines, each containing three numbers, namely: the value of n; the value of f_n computed by the iteration; and the value of f_n computed by the explicit expression, so the first few lines should look like this:

```
0 1 1
1 1 1
2 2 2
3 3 3
4 5 5
5 8 8
```

(And, yes, we will check that you're not just outputting the same number twice!)

Upload your well-documented driver code, `analysis_exercise0.cpp` and the pdf file containing your results, `analysis_exercise0.pdf` (which must have **no more than five pages** incl. the title page which must show your name and student ID) to the course's GRADESCOPE page, accessible via Canvas. You can upload your files as many times as you want. The grade for the exercise (out of a maximum of zero!) will be based on the final version received before the submission deadline of February 31, 3024.

Grades will be awarded based on the following criteria:

1. GRADESCOPE**-based assessment of code:**

   Up to 2 marks: Does the code produce the required results?

28/09/2026, 07:59:00

2. **Assessment of coding:**

   2 marks: Clear, well-documented and efficient code (no unnecessary steps or wasteful memory usage) with sensibly chosen variable/function names. Good, clear layout.

   1 mark: Code suffers from a significant number of these: inefficient (structure and memory usage), badly organised, unintuitive variable names, poor or inconsistent comments.

   0 marks: No code, or code that is a contender for the obfuscated C++ contest[a]. Code that doesn't even compile, or code that produces the results but not by the requested method.

3. **Assessment of report:**

   2 marks: Concise report providing all the requested information; good layout and font size; clear and well-annotated, self-explanatory plots (where plots were requested)

   1 mark: Report suffers from a significant number of these: doesn't provide all the requested information; poor layout; too much information (particularly if this is achieved by using an overly small font size); poor plots (in terms of layout, annotation and readability).

   0 marks: No report or a report that suffers from any of these: doesn't respect the page limit; doesn't address the questions and/or is hard to follow; extremely poor (or no) plots (where plots were requested).

Each Analysis Exercise is worth 2.5% of the overall grade for the course. However, the main intention is for these exercises to be "formative", i.e. they're supposed to give you practice, and to help you identify areas you can improve before working on the "big" projects. You should therefore compare what you've done with our model solutions (published shorty after the submission deadline for each Analysis Exercise) and ask for advice and more detailed feedback during the computer lab classes.

[a] Sadly, that doesn't exist, but there is a C version: `https://www.ioccc.org/`.

And after all this background, here's the real execise for this week:

Analysis exercise 1: The logistic map

- Write a program that displays the first 51 values of x_n, defined recursively via

$$x_{n+1} = \alpha\, x_n(1 - x_n), \qquad x_0 = 1/2,$$

  where α is a constant parameter. The code you submit to GRADESCOPE must output (on the screen, using `std::cout`) 51 lines, each containing the value of n and x_n, for $n = 0, ..., 50$ for $\alpha = 3.2$.

- Run your program for $\alpha = 1.5$, $\alpha = 3.2$, $\alpha = 3.5$ and $\alpha = 3.62$.

- Generate graphs of your output by plotting the value of x_n against n for these

28/09/2026, 07:59:00

four values of α. How do the successive iterates behave as $n \to \infty$? Describe the patterns you observe.

- You should find that the behaviour for $\alpha = 1.5$ is particulary simple. Determine an explicit analytical prediction for $x_\infty = \lim_{n\to\infty} x_n$ and check that it agrees with the computational result.

Upload your well-documented driver code, `analysis_exercise1.cpp` and the pdf file containing your results, `analysis_exercise1.pdf` to the course's GRADESCOPE page, accessible via Canvas. The pdf file of your report must have **no more than four pages**, incl. the title page which must show your student ID.
You can upload your files as many times as you want. The grade for the exercise will be based on the final version received before the submission deadline:

14:00 on Tuesday of Week 3.

28/09/2026, 07:59:00